

CH HIVE

- ☐ Introduction
- ☐ Architecture
- ☐ Data type
- ☐ Data model
- ☐ Databases
- ☐ Tables
- ☐ Data manipulation
- ☐ Hive queries



Introduction



- Hadoop has become an essential tool for processing massive data (data warehouses, enterprise data storage, etc.).
- Problem: **MapReduce is a very low-level language** → i.e. very close to the hardware → a developer needs to know how to interact with the hadoop cluster → not so simple, especially for business users.
 - An abstraction language that is close to the user and expresses business requirements in the simplest possible way
- **Hive** : Is an abstractive langage
 - * designed for business users (without needing to know the low-level hardware: Hadoop cluster)
 - * Allows MapReduce jobs to be expressed in a way similar to SQL query language (very familiar to database management business users)
 - * Originally designed by FaceBook for its analysts and data scientists to exploit huge volumes of data

Introduction



Abstraction level

Completely abstract language
close to the user. Designed for
business users

Closer to the user than to the
machine

Closer to the user than to the
machine

Low-level language very close to
hardware

SQL

HiveQL

Pig Latin

Hadoop (MapReduce)

Introduction



Word count en Java

```
package org.myc;

import java.io.IOException;
import java.util.*;

import org.apache.hadoop.fs.Path;
import org.apache.hadoop.conf.*;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;

public class WordCount {

    public static class Map extends Mapper<LongWritable, Text, Text,
    IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(LongWritable key, Text value, Context context) throws
        IOException, InterruptedException {
            String line = value.toString();
            StringTokenizer tokenizer = new StringTokenizer(line);
            while (tokenizer.hasMoreTokens()) {
                word.set(tokenizer.nextToken());
                context.write(word, one);
            }
        }
    }
}
```

```
public static class Reduce extends Reducer<Text, IntWritable, Text,
IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values, Context
context)
        throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum));
    }
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();

    Job job = new Job(conf, "wordcount");

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    job.setMapperClass(Map.class);
    job.setReducerClass(Reduce.class);

    job.setInputFormatClass(TextInputFormat.class);
    job.setOutputFormatClass(TextOutputFormat.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    job.waitForCompletion(true);
}
```

Introduction



Word count en HiveQL

```
CREATE TABLE docs (line STRING);

LOAD DATA INPATH 'docs' OVERWRITE INTO TABLE docs;

CREATE TABLE word_counts AS
SELECT word, count(1) AS count FROM
    (SELECT explode(split(line, 's')) AS word FROM docs)
GROUP BY word
ORDER BY word;
```



■ What is Hive

- Hive is a data storage infrastructure based on Hadoop.
- Hive is designed to facilitate data synthesis, ad-hoc querying and analysis of large volumes of data.
- Hive provides a simple query language called HiveQL, which is based on SQL and allows users familiar with SQL to ad-hoc query, summarize and analyze data easily.
- HiveQL allows users to integrate **Mapreduce plug-in** to be able to plug in their custom mappers and reducers to do more sophisticated analysis that may not be supported by the built-in capabilities of the HiveQL language.

Introduction



■ What Hive isn't good at

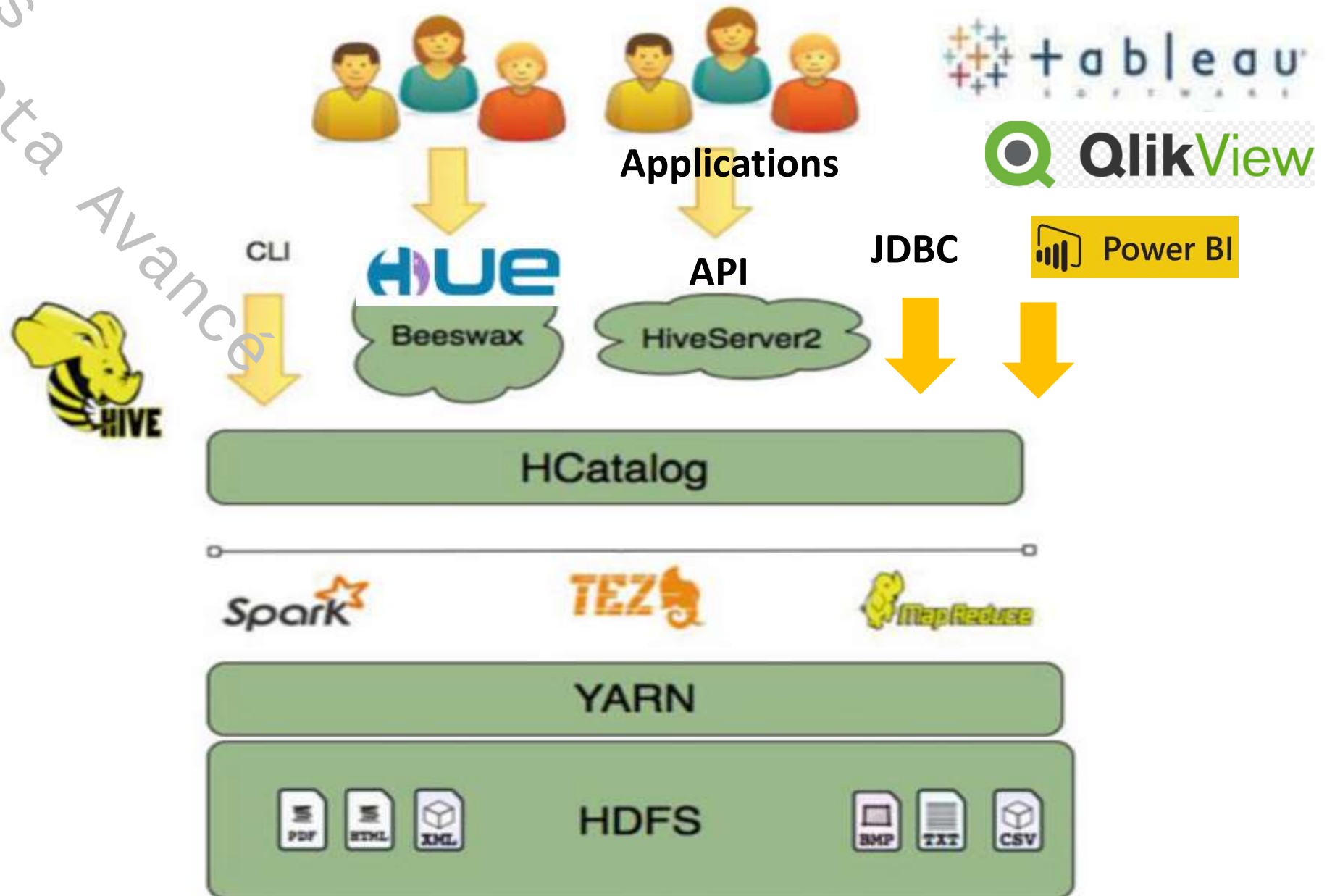
- Hadoop is a batch processing system and Hadoop jobs tend to **have high latency**
- As a result, the latency of Hive queries is generally **very high (minutes)** even for small data sizes (hundreds of megabytes) → As a result, Hive cannot be compared with systems such as Oracle when analyses are performed on a much smaller amount of data.
- Hive aims to provide acceptable (but not optimal) latency for interactive data browsing
- Hive is **not designed for online transaction processing** and does **not offer real-time queries** or online level updates.
- Hive **is adequate for batch** jobs on large immutable datasets (e.g. web logs).

Hive : Architecture

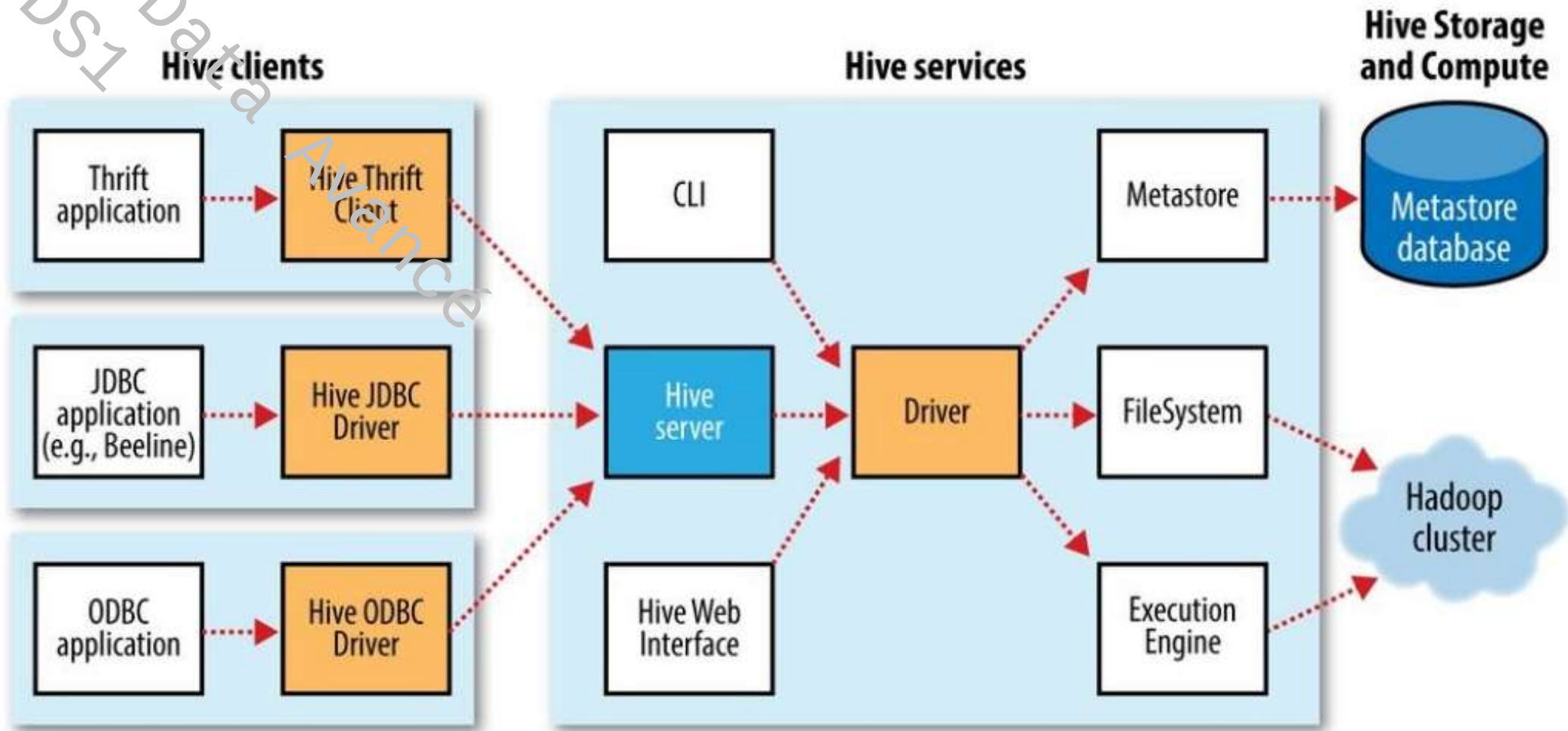


- Hive is not a standalone tool. It relies on various components to store and query data.
- In the Hadoop ecosystem, Hive is seen as a client tool for accessing data.
- Data access requires resources: calculation, storage, management, etc.
- Hive is similar to SQL, but it is not SQL, especially when it comes to processing speed.
- Apache Hive **translates** programs written in **HiveQL** (SQL-like) language into one or more **MapReduce, Tez or Spark** tasks (three execution engines that can be launched on Hadoop YARN).
- Mapreduce, Tez and Spark are batch processes, whereas SQL is an interactive processing language.

Hive : Architecture



Hive : Architecture





■ Hcatalog

- Hcatalog: generally considered to be a separate component of Hive
- Hcatalog and Hive are inseparable.
- When you create a Hive table, you create a structure in Hcatalog.
- Hcatalog facilitates the sharing of schemas between different Hadoop components.
- advantages :
 - ✓ Provides a **common schema environment for multiple tools**
 - ✓ Allows tool connectors to read data from and write data to the Hive repository
 - ✓ Allows users to **share data between tools**
 - ✓ **Creates a relational structure** for Hadoop data
 - ✓ **Abstraction** of data management (access/location)

Hive : Architecture - HCatalog



- The Hadoop software infrastructure is designed to store data of various kinds (structured, semi-structured, unstructured) → no reading schema, unlike relational databases where data is structured in tables (rows and columns).
- → Hive relies on a data storage software layer installed underneath the execution engine (MapReduce, Tez, Spark), like Hcatalog.
 - Not for storing data in HDFS
 - **BUT for storing user request metadata.**
 - just refers to pointers to data on the HDFS.
- Before launching HiveQL queries, a table (read schema) must be created in the metadatabase (Metastore). HCatalog tables are relational tables → abstraction of HDFS read/write → A language similar to SQL can be used (SQL-Like) → HiveQL

Hive : Architecture - Hiveserver2



- Software applications are written using a specific programming language (Python, Java, C#, etc.).
- OR databases are only accessible via query languages (e.g. SQL).
- **Problem:** connection between software application and database
- **Solution:** translation interface between the two languages (application and database)
 - ✓ ODBC (Open Database Connectivity)
 - ✓ JDBC (Java Database Connectivity)
 - ✓ Thrift client (call a piece of code in a remote process)
- ODBC: does not depend on a specific programming language, database system or operating system
- JDBC: is an API for the JAVA language. It is an interface that allows a client to access a DBMS. JDBC is best suited to object-oriented databases. Any ODBC-compatible database can be accessed using the JDBC-to-ODBC bridge.

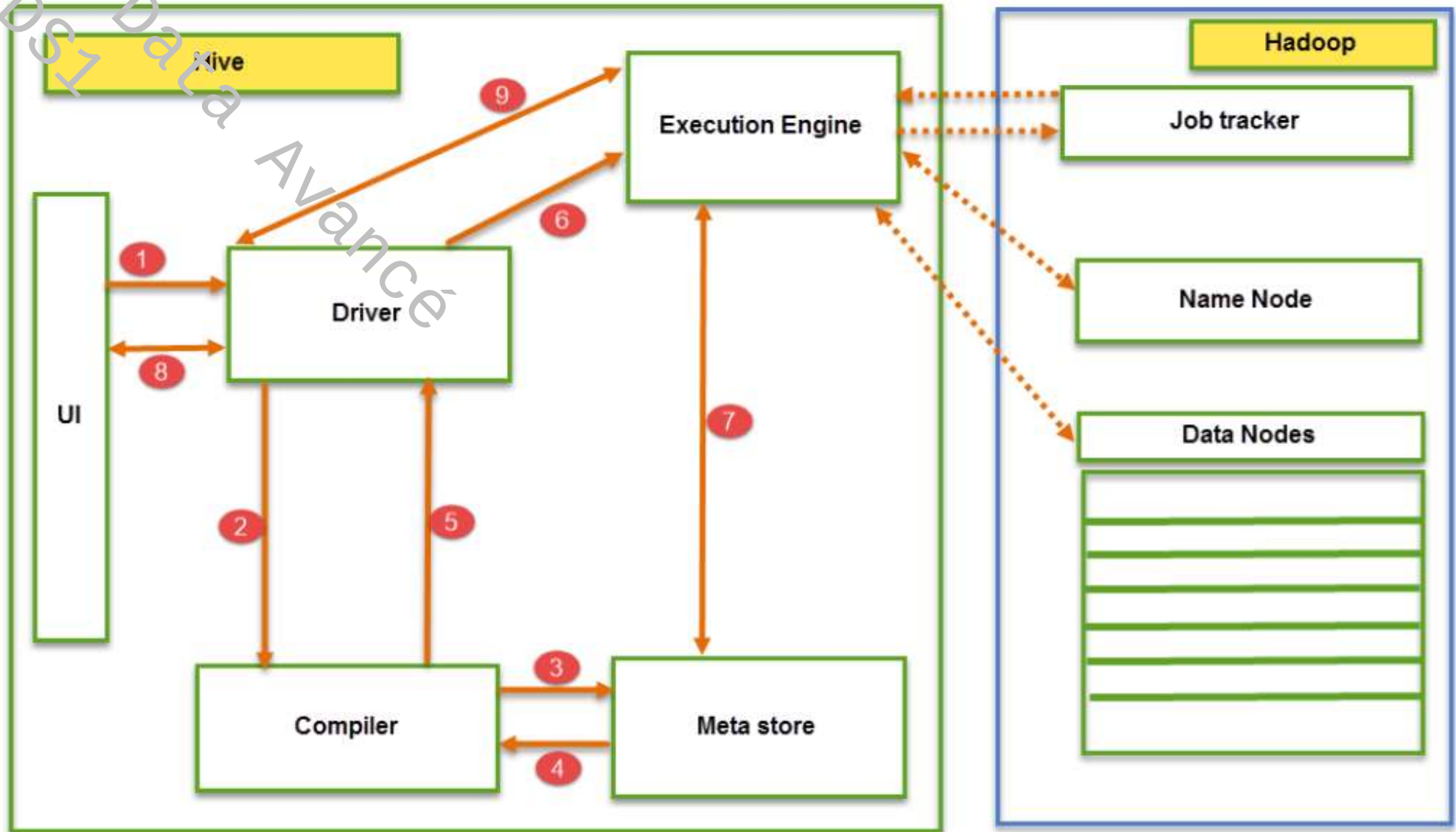


- There are several ways to interact with Hive.
 - CLI: command-line interface.
 - Graphical interface: several projects (commercial / open source)
 - * Ambari
 - * Karmasphere
 - * Cloudera open source Hue (<https://github.com/cloudera/hue>),
 - * Qubole (<http://qubole.com>) and others.
 - Thrift, JDBC, ODBC
 - HiveServer APIs that enable applications to interact with Hive using REST procedure calls or requests.
 - Business Intelligence (BI) and data visualisation tools (Tableau, QlikView, PowerBI, etc.)



- Driver: this is the component that manages requests. It is responsible for orchestrating the lifecycle of a request by invoking the necessary components in the right order.
- Driver manages sessions and keeps track of statistics.
- Driver: three main components
 - ✓ *Compiler: checks the syntactic and semantic correctness of requests, then generates a physical plan in the form of a DAG.*
 - ✓ *Optimizer: optimises the plan generated by the compiler.*
 - ✓ *Executor: executes the queries on the Hadoop cluster. It is the interface between the Hadoop cluster and Hive.*

Hive : Architecture - Fonctionnement





- Step 1: executeQuery: the user uses an interface (CLI, web, Thrift) to launch a query
- Step 2: getPlan: Driver accepts the request, creates a session manager for the request and sends the request to the compiler to generate the execution plan.
- Step 3: getMetaData: The compiler asks the MetaStore to send it the metadata.
- Step 4: sendMetaData: The MetaStore sends the metadata to the compiler. The compiler uses this metadata to perform type checking and semantic analysis. The compiler then generates the DAG (Directed acyclic Graph) execution plan.
- Step 5: sendPlan: The compiler then sends the generated execution plan to the Driver.
- Step 6: executePlan: After receiving the execution plan from the compiler, Driver sends the execution plan to the execution engine to execute the plan.



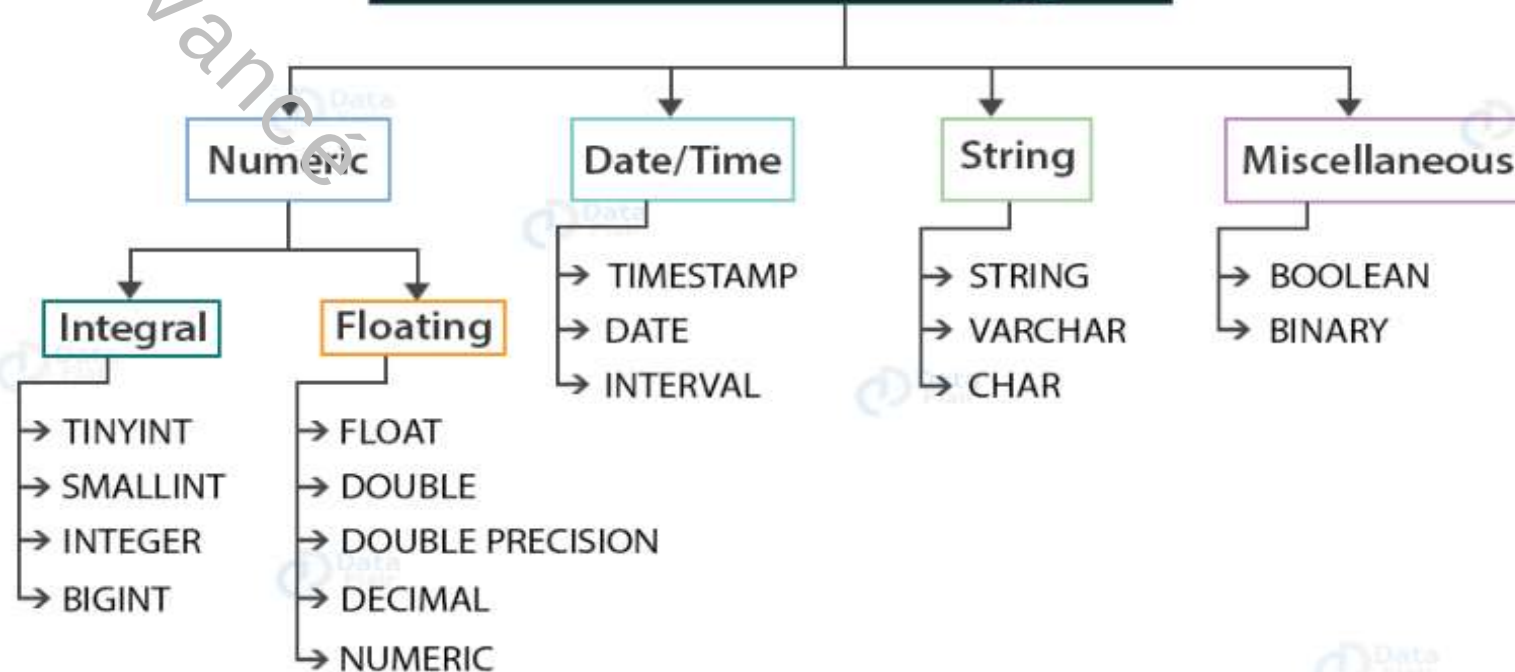
- Step 7: Execution Engine (EE) acts as a bridge between Hive and Hadoop to process the request.
 - ✓ EE must first contact NameNode, then the DataNodes to access the data according to the tables defined in the MetaStore (EE retrieves the desired data from the DataNodes. The actual table data resides only in the DataNodes. From the NameNode, it retrieves only the metadata for the query.
 - ✓ EE then collects the actual data relating to the HiveQL query from the DataNodes.
 - ✓ EE communicates bidirectionally with MetaStore in Hive to perform DDL operations (Data Definition Language. E.x. CREATE, DROP and ALTERING...).
 - ✓ EE in turn communicates with Hadoop daemons such as NameNode, Data nodes and jobtracker to execute the request on the Hadoop file system.
- Step 8: Retrieving driver results
- Step 9: Sending results to EE

Hive : Data Types



- Data types in Hive specify the type of column/field in the Hive table → specifies the type of values that can be inserted in the specified column

Primitive Data Types



Complex Data Types



Hive : Data types



Numeric Types:

Type	Memory allocation
TINY INT	Its 1-byte signed integer (-128 to 127)
SMALL INT	2-byte signed integer (-32768 to 32767)
INT	4 -byte signed integer (-2,147,484,648 to 2,147,484,647)
BIG INT	8 byte signed integer
FLOAT	4 - byte single precision floating point number
DOUBLE	8- byte double precision floating point number
DECIMAL	We can define precision and scale in this Type

String Types:

Type	Length
CHAR	255
VARCHAR	1 to 65355
STRING	We can define length here(No Limit)

Miscellaneous

✓ BOOLEAN

True

False

✓ BINARY

tableau d'octets

Timestamp : Time stamping
Supports traditional Unix time stamping with optional nanosecond accuracy
Date : format "YY-MM-DD"
Ex. 2020-04-22 11:22:30

Hive : Data Types



ARRAY : Ordered sequences of the same type that are indexed starting from zero.

Ex. classe ARRAY<'MPDS1', 'Première Année'>
classe[0] → MPDS1

MAP : collection of key/value pairs. Accessible by key

Ex. details map<'institution','FSB', 'niveau', 'mastère', 'spécialité', 'Data Science' >
details('institution') → 'FSB'

STRUCT : similar to the structure in C. A structure is an object that contains various fields that can be of any data type, unlike ARRAY (same type).

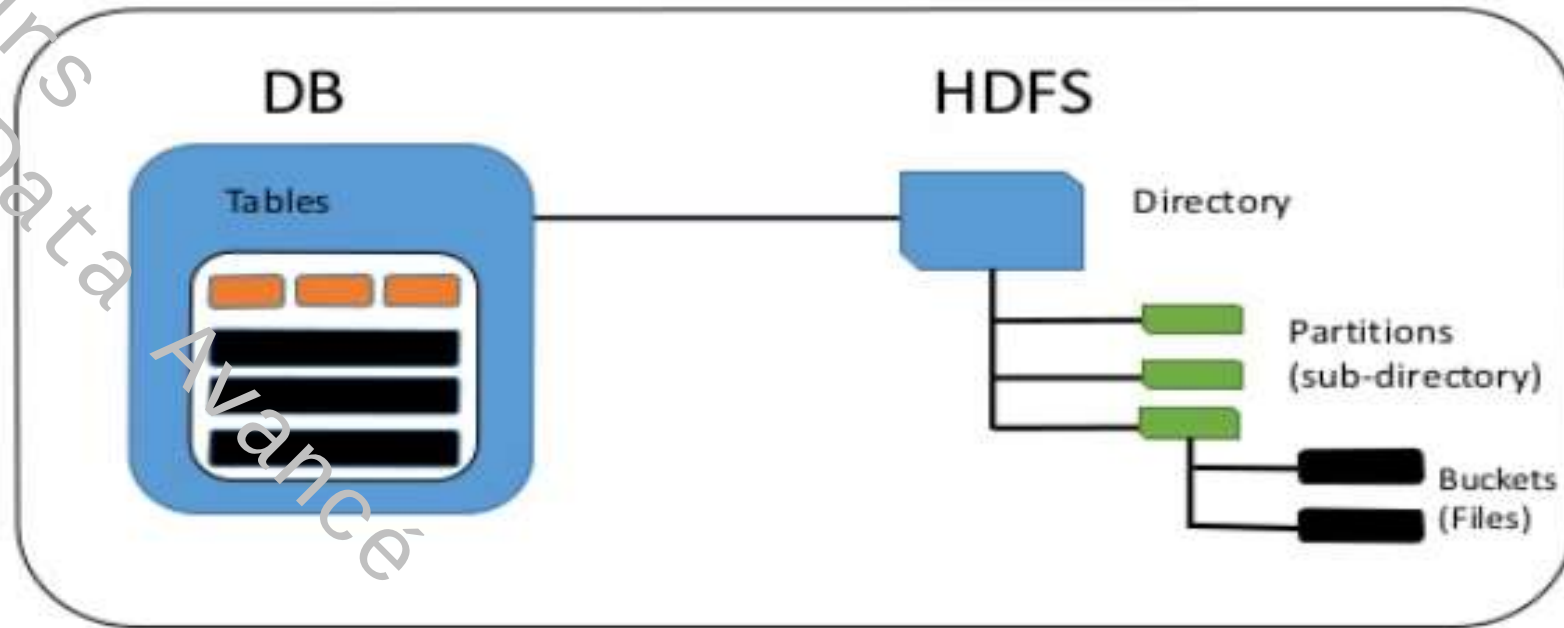
EX. adresse STRUCT <Governorat:STRING, ville:STRING, codeZIP:INT>
adresse <'Bizerte', 'Zarzouna', 7021>
adresse.ville → 'Zarzouna'

Hive : Data Model



- Two considerations
 - **Storage space:** Hive is an open source data warehouse system built on top of Hadoop to query and analyse the large datasets stored in Hadoop files.
 - **Types of data:** different types
 - Structured
 - Semi-structured
 - Unstructured
- Data model (structuring & access) :
 - **A schema**
 - tables (columns, rows),
 - partitions
 - bucket
 - These objects are logical units defined in the metadata layer (Hive Metastore).
 - The Hive schema means that the data is seen in 02 dimensions as familiar SQL tables, regardless of their structure.

Hive : Data Model



■ DB : Data Base

- The Hive concept of a database is a catalogue (table namespace)
- Useful for large clusters with several teams and users → a way of avoiding table name collisions.
- Allows production tables to be organized into logical groups.

Hive : Database - creation and basic commands



- If the database is not specified, the default database directory is used (default).
- Hive creates a directory for each database
- Creation : **DATABASE** or **SCHEMA**
hive> **CREATE DATABASE mpds1;**
- List the databases
hive> **SHOW DATABASES;**
default
fsb
mpds1
mrsi1
- Search for database (regular expression)
hive> **SHOW DATABASES LIKE 'f.*';**
fsb
hive> ...

Hive : Database - creation and basic commands



- The database directory is created under a top-level directory defined by the: **hive.metastore.warehouse.dir.**
- Default value : **/user/hive/warehouse**
- Ex. fsb database is created in **/user/hive/warehouse/fsb.db**
- It is possible to choose other directory

```
hive> CREATE DATABASE fsb
      > LOCATION '/mon/repertoire/prefere';
```

- Add description (comment) :

```
hive> CREATE DATABASE mpds1
      > COMMENT 'suivie d'enseignement à distance de MPDS1';
```

```
hive> DESCRIBE DATABASE mpds1;
mpds1 suivie d'enseignement à distance de MPDS1
hdfs://master-server/user/hive/warehouse/mpds1.db
```

RQ : An instruction ends with ";" → several lines for a single instruction

Hive : Database - creation and basic commands



- More information : key-values
- A way of adding information to the output
- DESCRIBE with the EXTENDED argument

```
hive> CREATE DATABASE mpds1  
      > WITH DBPROPERTIES ('niveau' = première année, 'date' =  
'2020-06-08');
```

```
hive> DESCRIBE DATABASE mpds;  
mpds hdfs://master-server/user/hive/warehouse/mpds.db
```

```
hive> DESCRIBE DATABASE EXTENDED mpds;  
mpdss hdfs://master-server/user/hive/warehouse/financials.db  
{date=2020-06-08, niveau=première année};
```

Hive : Database - creation and basic commands



- Change current directory

hive> USE fsb;

→ Now, commands such as SHOW TABLES ; will display the list of tables in this database.

- Delete database

hive> **DROP DATABASE IF EXISTS** fsb;

The IF EXISTS option is optional and suppresses warnings if the fsb database does not exist.

- By default, Hive does not allow to delete a database if it contains tables. We must first delete the tables or add the CASCADE keyword to the command (Hive deletes the tables in the database first).

hive> **DROP DATABASE IF EXISTS** fsb **CASCADE**;



► Tables

- The Hive data model provides a logical row/column view of data called a table.
- Similar to a relational database, a Hive table consists of a two-dimensional definition of the data.
- However, the data exists independently of the table.
- The data in a Hive table exists in an HDFS directory and the table definition (metadata) is stored in a relational database (Hcatalog).

■ SGBD Vs Hive

- In relational databases, deleting a table deletes its definition and the underlying data.
- In Hive, if you define a table as an external table, the definition of the table will be independent of the data → deleting the table **does not delete the data**
- The same data can be associated with multiple tables
- Hive → supports multiple data types (unlike DBMS)



- Hive has 02 types of tables :
 - Internal or Managed Tables**
 - External Table**

- External Table

External tables are created using EXTERNAL keywords with the CREATE TABLE statement.

- This is the recommended table type for all production deployments of Hadoop.
- In the case of external tables, Hive does not delete the data from the file system as if the table were deleted.

- Internal Table

- An internal table in Hive refers to a table whose data is managed by Hive.
- If you delete an internal table → Hive will also delete its underlying data.
- These tables are not used very often in Hadoop



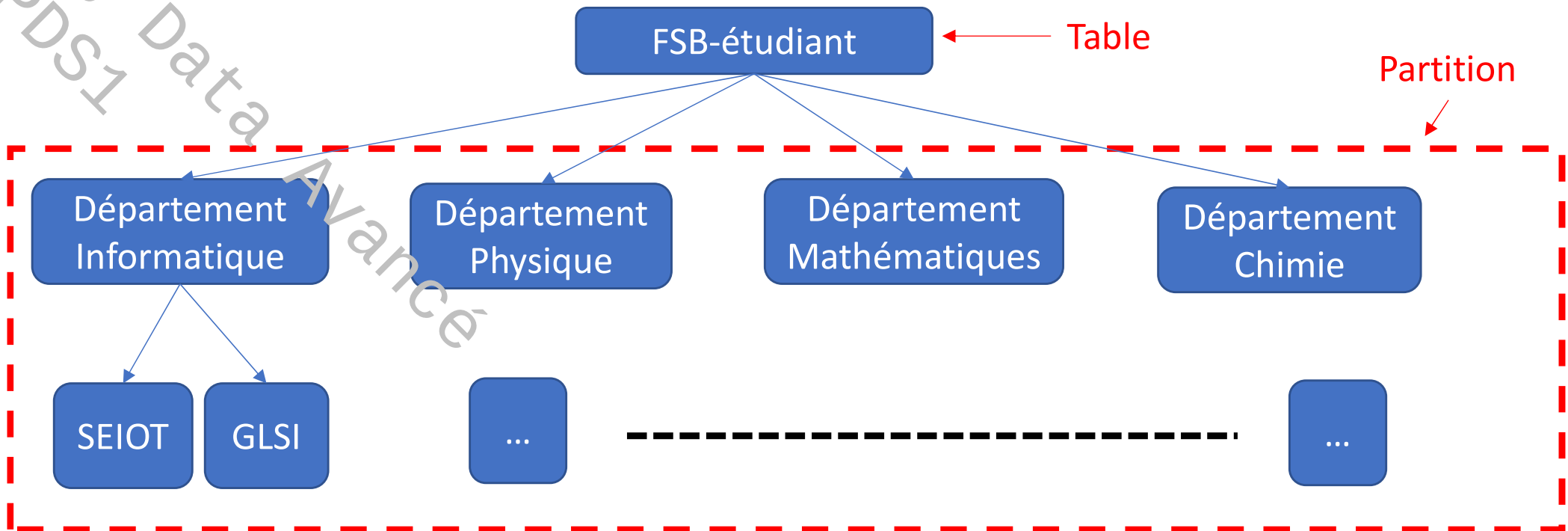
■ Partition

- Used to distribute the load horizontally by physically moving the data as close as possible to its most frequent users.
- Hive has the notion of table partitioning
 - ✓ Performance
 - ✓ Logical organisation of data (hierarchical)

■ Ex.

- Let's go back to the example of the Faculty of Science in Bizerte
- Our administrators launch queries with filtering (WHERE) to select particular students: department, section, level, ...

```
hive> CREATE TABLE fsb (  
  > nom STRING, age INTEGER, genre STRING  
  > departement STRING, section STRING, niveau INTEGER  
  > adresse STRUCT<rue:STRING, ville:STRING, zip: INTEGER  
  > )  
  > PARTITIONED BY (departement STRING, section STRING);
```



- Table partitioning changes the way Hive structures data storage.
- Let's assume that the table is created in the madb database
hdfs://master_server/user/hive/warehouse/madb.db/fsb
→ Hive will now create sub-directories reflecting the partitioning structure.

Hive : Data Model



- Hierarchical directories created
 - .../fsb/departement=informatique/section=SEIOT
 - .../fsb/departement=informatique/section=GLSI
 - ...
 - .../fsb/departement=chimie/section=chimie industrielle
- Each directory & sub-directory contains 0 or more files
- Once the table has been created, the partition keys behave like columns in a relational database.
- Ex.

SELECT * FROM fsb

WHERE departement = 'Informatique' **AND** section = 'SI';

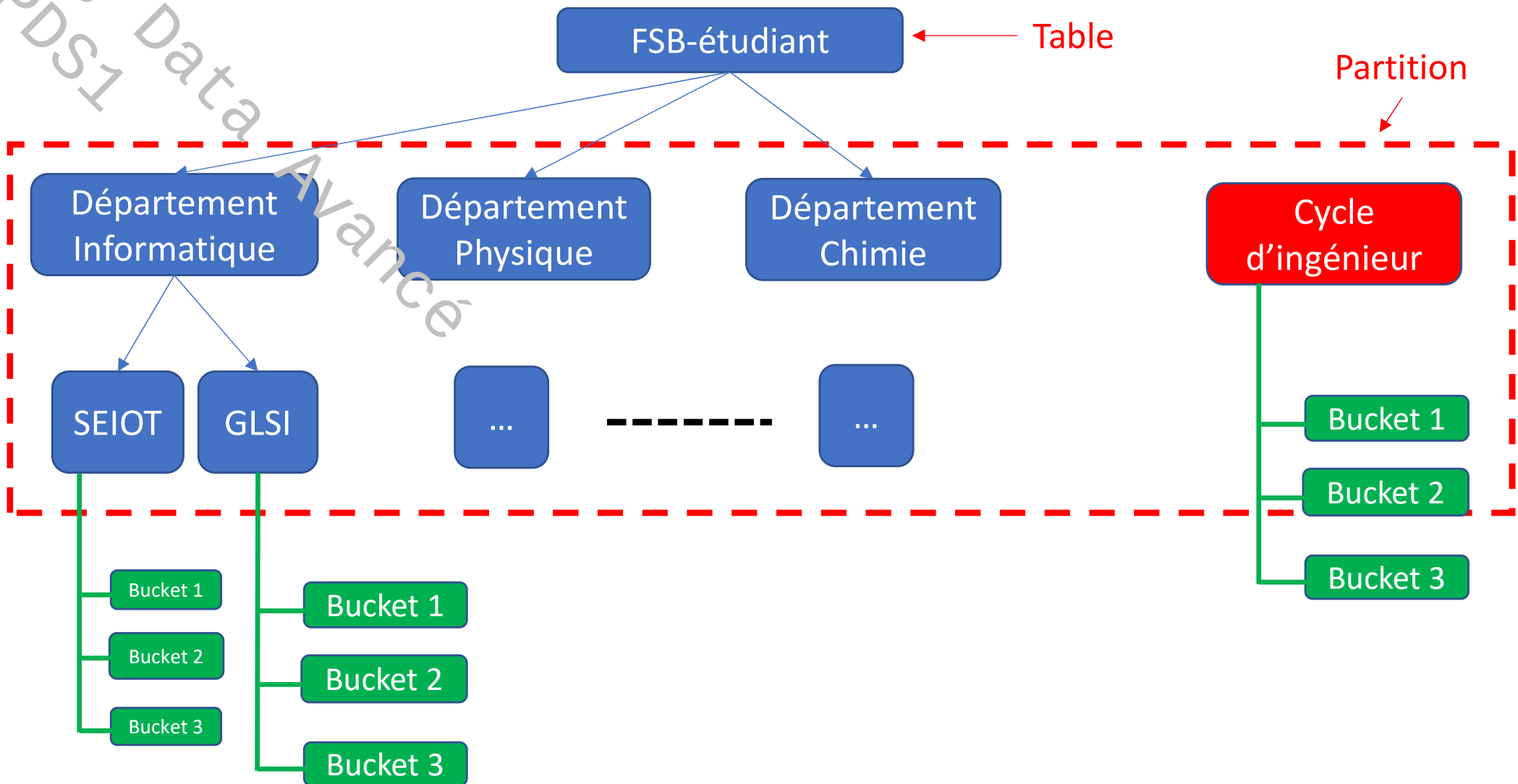
→ select all students in the IS section of the IT department

Note: In reality, we could have run the same query without first going through the partition. However, for large databases, which is the case with Big Data, this will take a lot of time → partition interest



► Bucket

- This is an additional tool for structuring data
- purpose: to improve the performance of HiveQL queries
- Allows you to subdivide tables and/or partitions
- Solves the problem of over-partitioning and allows data to be broken down into more manageable sets of files
- Over-partitioning :
 - Let's go back to the example of the fsb table.
 - Let's suppose we've just created a new department (data science engineering course). The number of students will be very limited and, as a result, the size of the file relating to this new department will probably be small compared to the other departments.
 - for reasons of query performance (response times), it is better to have files of balanced sizes.
 - Bucket gives users the option of subdividing large files into several smaller ones.





- The Boolean variable (hive.enforcing.bucketing= TRUE) must be set to TRUE.
- Buckets are created using a Hashing function which uses a bucket key to create buckets.
- The key must be chosen carefully to avoid the skewness problem associated with data distribution (choose a key with a large number of distinct values) (take note of the table).
- Ex. table fsb

```
hive> CREATE TABLE fsb (  
  > name STRING, age INTEGER, genre STRING  
  > departement STRING, section STRING, niveau INTEGER  
  > adresse STRUCT<rue:STRING, ville:STRING, zip: INTEGER  
  > )  
  > PARTITIONED BY (departement STRING, section STRING);  
  > CLUSTERED BY (age) INTO 3 BUCKETS;
```



■ Creation

- CREATE TABLE statement.
- Hive's version of CREATE TABLE is quite similar to standard SQL. However, it provides various options to add to the versatility of managing the various types found in the Big Data world.
- Data is of different types (not all structured in rows-columns) □ Configuration specified when creating a table which defines how Hive will interpret the data
- Hive has built-in data interpreters that allow Hive to interpret several types of data (the most common): called Serdes (Serialisation Deserialisation).
- Hive also allows you to define your own Serdes, and all you have to do is plug them into a CREATE TABLE statement to enable Hive to understand the data format.

Hive : Table



Exemple

```
CREATE TABLE IF NOT EXISTS mydb.MPDS1 (  
  NomPrenom STRING COMMENT 'nom et prénom de l'étudiant' ,  
  moyenne FLOAT COMMENT 'la moyenne' ,  
  matieres ARRAY<STRING> COMMENT 'liste de matières' ,  
  moyMatiere MAP<key:STRING, values:FLOAT> COMMENT 'key : la matière, values : moyenne par matière' ,  
  adresse STRUCT<rue:STRING, ville:STRING, gouvernorat:STRING, zip:INT>  
  COMMENT 'Home address' )  
  
COMMENT 'Description de la table MDS1'  
TBLPROPERTIES ('créateur'='moi', 'créé_at'='2020-06-10 10:00:00', ...)  
  
LOCATION '/user/hive/warehouse/mydb.db/MPDS1' ;
```



- IF NOT EXISTS:** By default, Hive checks whether a table with the same name exists at the time of creation. If so, Hive generates a warning. **IF NOT EXISTS** allows you not to generate a “warning” and to create a table with an existing name by overwriting the old table.
- mydb.MPDS1: You can specify the location of the table by attaching the created table to a database using (.)
 - You can add a comment to any column, after the type.
 - As with databases, a comment can be attached to the table itself.
 - The main advantage of **TBLPROPERTIES** is that it allows you to add additional documentation in a key-value format.
 - You can optionally specify a location for the table data.

Hive : Table



- Copy the schema of the table
 - It is possible to copy the schema (but not the data) of an existing table:

```
CREATE TABLE IF NOT EXISTS mydb.MPDS2  
LIKE mydb.MPDS1;
```

- List the tables in a database
 - Current database :
hive> USE mydb;
hive> SHOW TABLES;
MPDS1
MPDS2
 - Different database
hive> USE default;
hive> SHOW TABLES IN mydb;
MPDS1
MPDS2

Hive : Table



- Regular expression
- Searching table

```
hive> USE mydb;
```

```
hive> SHOW TABLES;
```

```
MPDS1
```

```
MPDS2
```

```
SI1
```

```
SI2
```

```
SI3
```

```
hive> SHOW TABLES 'MPDS.*';
```

```
MPDS1
```

```
MPDS2
```



- Display comments and details of tables
- Use the **DESCRIBE EXTENDED** mydb.MPDS1 command to display details of the table.
- We can delete the mydb prefix if we are currently using the mydb database.
`hive> DESCRIBE EXTENDED mydb.MPDS1;`
- We can display comments of one column
`hive> DESCRIBE mydb.MPDS1.moyenne;`
`moyenne float    la moyenne finale de l'étudiant`



► Reading files

- The following statement creates an external table that can read all comma-delimited data files in **/data/stocks** (**external table**):

```
CREATE EXTERNAL TABLE IF NOT EXISTS stocks (  
  exchange STRING,  
  symbol STRING,  
  ymd STRING,  
  price_open FLOAT,  
  price_high FLOAT,  
  price_low FLOAT,  
  price_close FLOAT,  
  volume INT,  
  price_adj_close FLOAT)  
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','  
LOCATION '/data/stocks';
```

- Because it is external, deleting the table does not delete the data, although the table's metadata is deleted.



■ Delete table

- The familiar **DROP TABLE** command from SQL is supported:
DROP TABLE IF EXISTS MPDS1;
- The IF EXISTS keywords are optional. If they are not used and the table does not exist, Hive returns an error.
- For managed tables, the table's metadata and data are deleted.

■ Modify table

- The **ALTER TABLE** command is used to modify the table
- ALTER TABLE modifies only the table metadata. The table data remains intact.
- Care must be taken to ensure that all modifications are consistent with the actual data.

Hive : Table



- Ex. rename table

```
ALTER TABLE log_messages RENAME TO logmsgs;
```

- Ex. Add partitions

```
ALTER TABLE logmsgs ADD IF NOT EXISTS  
PARTITION (year = 2011, month = 1) LOCATION '/logs/2011/01'  
PARTITION (year = 2011, month = 2) LOCATION '/logs/2011/02'  
PARTITION (year = 2011, month = 3) LOCATION '/logs/2011/03';
```

- Ex. Eliminate partitions

```
ALTER TABLE logmsgs DROP IF EXISTS PARTITION(year = 2011, month = 2);
```

- Ex. Add columns

```
ALTER TABLE logmsgs ADD COLUMNS (  
app_name STRING COMMENT 'Application utilisée';
```

- *Note: Consult the official website <http://hive.apache.org/>*
- *<https://cwiki.apache.org/confluence/display/Hive/Tutorial>*

Data handling - Loading Data



■ Loading data into table

- Since Hive has no insert, update or delete operations at table row level, the only way to put data into a table is to use one of the load operations.
- Or you can simply write the files to the correct directories by other means (you can copy a file to the specified location using the HDFS **put** or **copy** commands and create a table pointing to that location with all the relevant row format information).
- Example

```
LOAD DATA LOCAL INPATH '/data/MPDS1'  
OVERWRITE INTO TABLE MPDS  
PARTITION (niveau = 'première_année');
```

- The command creates a partition (if it does not exist) and copies the data to the partition created
- **LOAD DATA LOCAL** ... copies local data to the final location in the distributed file system, while **LOAD DATA** ... (i.e. without **LOCAL**) moves the data to the final location.

Data handling - Loading Data



- Syntax

LOAD DATA [LOCAL] INPATH 'filepath' [OVERWRITE] INTO TABLE tablename

- **LOAD DATA**: keyword to download the data into Hive.
- **LOCAL** if used, allows data to be downloaded from the local directory. If omitted, data is downloaded from the directory specified by the **fs.default.name** variable.
- **INPATH 'filepath'**
 - if LOCAL is used: file:///user/hive/example
 - if LOCAL is omit : dfs://namenode:9000/user/hive/example
- **OVERWRITE**: downloads the data into an existing table and overwrites the existing data. If OVERWRITE is omitted, the existing data is augmented by the new data.
- **INTO TABLE**: Table in Hive



Insertion with Hive Query

- Commonly used operation
- Insert data into tables (intermediate or final) from Hive queries launched on existing tables and which reflect data processing
- **OVERWRITE**: replace existing data in a table with the result of a query

INSERT OVERWRITE TABLE etudiant2 **SELECT** id, nom, age, moyenne **from** etudiant1;

- **INTO** : Extending the existing table

INSERT INTO TABLE etudiant2 **SELECT** id, nom, age, moyenne **from** etudiant1;

Data handling - Loading Data



- Creating a table and inserting data (a single query)

```
CREATE TABLE MPDS1
AS SELECT nom, prenom, moyenne
FROM etudiant_informatique ei
WHERE ei.section = 'MPDS1';
```

RQ: the table schema created is defined by **AS**

- Inserting a row directly into a table

```
INSERT INTO TABLE MPDS1 values('Salem', 'Salem', 17.28);
```

- Exporting data from a Hive table

```
INSERT OVERWRITE LOCAL DIRECTORY '/tmp/MPDS1_etudiant'
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'
SELECT * FROM MPDS1;
```

Hive query - Select



- Let's take the example of slide 37 again

- SELECT : projection operator

```
hive> SELECT NomPrenom, moyenne FROM MPDS1;  
Salem salem 12.42  
Marnissi Khouloud 13.65
```

- Avec alias

```
hive> SELECT m.NomPrenom, m.moyenne FROM MPDS1 m;  
Salem salem 12.42  
Marnissi Khouloud 13.65
```

- It is possible to access a specific field of a complex structure in a table (matieres is a complex type of ordered ARRAY data → accessible by position).

```
hive> SELECT m.NomPrenom, m.matieres[0] FROM MPDS1 m;  
Salem Salem BigData
```

Hive query - Select



- Selection of a specific STRUCT field (a STRUCT field can be accessed by (.))

```
hive> SELECT NomPrenom, adresse.ville FROM MPDS1;  
Salem Salem NouakChott  
Saada Islem Bizerte
```

- Selecting a specific field in a MAP structure

```
hive> SELECT NomPrenom, moyMatiere["BigData"] FROM MPDS1;  
Salem Salem 14.3  
Saada Islem 11.5
```

- Specify columns with regular expressions
moyenne (FLOAT) and moyMatiere (MAP)

```
hive> SELECT NomPrenom, `mo*` FROM MPDS1;  
Salem Salem 12,36 {BigData:14.3, DataViz:10.5, ML:13.6}  
Saada Islem 12.24 {BigData:11.5, DataViz:13.5, ML:11.5}
```

Hive queries - operators



- Operation and calculation with columns

- Function

```
hive> SELECT upper(NomPrenom), round(moyenne) FROM MPDS1;  
SALEM SALEM 12  
MARNISSI KHOULOU 14
```

- Computing

```
hive> SELECT * FROM MPDS1 WHERE moyenne >= 12;
```

```
hive> SELECT * FROM MPDS1  
WHERE moyenne >= 12 && moyMatiere["BigData"] >10;
```



Fonctions

Round() → entier le plus proche

Floor() → partie entière

Ceil() → partie entière +1

exp(), ln(), log10(), log2(), ...

Power(x,n) → x^n

Sqrt()

Sin(), cos(), tan()

....

Opérateurs binaires (bit par bit)

$A \& B$

$A | B$

$A \wedge B$ (XOR)

$\sim A$ (NOT)

Opérateurs Arithmétiques

$A + B$

$A - B$

$A * B$

A / B

$A \% B$ (modulo)

Opérateurs relationnels

$A = B$

$A \neq B$

$A > B$, $A \geq B$

$A < B$, $A \leq B$

A is NULL, A is NOT NULL

A LIKE B

Opérateurs logique

$A \&\& B$ (AND)

$A || B$ (OR)

$!A$ (NOT)

Hive queries - aggregation



■ Aggregation

- A special type of function is the aggregate function, which returns a single value resulting from certain calculations on many lines.
- Ex.

```
hive> SELECT count(*) avg(moyenne) FROM MPDS1;
```

10 13.69

Nombre des
étudiant

La moyenne des moyennes des étudiants

Fonctions d'agrégation

Count(*), count(expression) → nombre de lignes (qui satisfont l'expression)

Count(DISTINCT col) → comptage des valeurs distinctes

Sum(col), avg(col), min(col), max(col), variance(col)

Corr(col1,col2),

....

Hive queries - CASE ... WHEN ... THEN



■ Condition : CASE ... WHEN ... THEN

```
hive> SELECT NomPrenom, moyenne,  
> CASE  
> WHEN moyenne < 10.0 THEN 'redouble'  
> WHEN moyenne >= 10.0 AND moyenne < 12.0 THEN 'Passable'  
> WHEN moyenne >= 12.0 AND moyenne < 14.0 THEN 'Assez Bien'  
> WHEN moyenne >= 14.0 AND moyenne < 16.0 THEN 'Bien'  
> ELSE 'Très Bien'  
> END  
> FROM MPDS1;
```

```
salem salem    12,36    Assez Bien  
Marnissi Khouloud  13.65  Assez Bien
```



■ Join

- Hive supports equality joins between tables to allow data from two tables to be combined.

- Syntax

```
SELECT table_fields
FROM table_one
JOIN table_two
ON (table_one.key_one = table_two.key_one
AND table_one.key_two = table_two.key_two);
```

Hive queries - join



- Exercise (example - Join) : Analyse the following code, identifying the role of each block.

```
USE MPDS1;  
CREATE TABLE MPDS1.listeEtudiant (  
  CIN int,  
  nom string,  
  prenom string  
)  
CLUSTERED BY (CIN) INTO 1 BUCKETS
```

```
INSERT INTO TABLE MDPS1.listeEtudiant  
VALUES  
(05002536, 'salem', 'salem'),  
(15669854, 'Marnissi', 'Khouloud'),
```

Hive queries - join



```
CREATE TABLE MPDS1.adresse (  
  CIN INT  
  ad string  
)  
CLUSTERED BY (CIN) INTO 1 BUCKETS
```

```
INSERT INTO TABLE MPDS1.adress2  
VALUES  
(05002536, 'NouakChott'),  
(15669854, 'Bizerte'),
```

```
SELECT listeEtudiant.CIN,  
listeEtudiant.nom,  
listeEtudiant.prenom  
FROM  
MPDS1.listeEtudiant  
JOIN  
MPDS1.adresse  
ON (listeEtudiant.CIN = adresse.CIN);
```

Exercise: Example of a Hive script



Exercice : Analyser le script Hive suivant en complétant les champs réservés aux commentaires afin de le rendre plus compréhensible

```
$HIVE_HOME/bin/hive
## .....
CREATE DATABASE census;
## .....
USE census;
## .....
CREATE TABLE person (
  persid int,
  lastname string,
  firstname string
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
## .....
LOAD DATA LOCAL INPATH 'file:///root/hive/example/person001' OVERWRITE INTO
TABLE person;
## .....
SELECT persid, lastname, firstname FROM person;
```

Exercice : Exemple d'un script Hive - Correction



```
$HIVE_HOME/bin/hive
## Create a new database
CREATE DATABASE census;
## Use the new database
USE census;
## Create a new table
CREATE TABLE person (
  persid int,
  lastname string,
  firstname string
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
## Load data into the new table from csv file
LOAD DATA LOCAL INPATH 'file:///root/hive/example/person001' OVERWRITE INTO
TABLE person;
## Check if the data is in table
SELECT persid, lastname, firstname FROM person;
```